

No Spoiler Please! A Distilled BERT Model to Detect Spoiler Reviews

Marco Colangelo^{1*}

¹Department of Computer Science, University of Milan, Milan, Italy.

Corresponding author(s). E-mail(s): marco.colangelo@studenti.unimi.it;

Abstract

This report contains the explanation of the work done for the final project of the Natural Language Processing course. The professor in charge of the course is Professor Alfio Ferrara, University of Milan. The project focused on the application of knowledge distillation, a technique used to compress large language models into smaller and more efficient versions. This is achieved by training a smaller model (the student) to replicate the behavior of a larger model (the teacher). In particular, this project had the aim to implement a spoiler detector for IMDb's movie reviews: a BERT-large model has been used as the teacher model, while a *bert_uncased_L-4_H-512_A-8* (*BERT small*) has been used as the student model. The experiments show that TO BE EXPENDED

1 Introduction

Very often, before even seeing a film, we decide to read some reviews because we're curious to know the opinions of those who have already seen it. In such a situation, it's not uncommon to encounter reviews that, whether intended to spoil something or not, include various types of spoilers (e.g., "The movie is great, but at the end, character X dies/does this or that"). Clearly, we want to avoid this, but platforms like IMDb don't warn us in advance about such reviews.

IMDb is by far the most popular global source for movies and TV shows, and it also includes user-submitted reviews. But, as mentioned above, there's no spoiler alert for reviews. What if there were a browser extension (running locally on users' PCs) that retrieves the text of a review and tells us "Spoiler Alert!" before we can even start reading it?

With the recent evolution of LLMs, this task can be accomplished by fine-tuning (adding a binary classification layer on top of the model) a sufficiently large pre-trained model, such as BERT-large.

Language model pre-training, such as BERT, has significantly improved the performances of many NLP tasks. However, pre-trained language models are usually computationally expensive, so it is difficult to efficiently execute them on resource-restricted devices (such as the user’s PC). A novel Transformer distillation method, specially designed for knowledge distillation of the Transformer-based models, allows to effectively transfer to a smaller student model called TinyBERT the plenty of knowledge from a large pre-trained BERT model. A two-stage learning framework for TinyBERT which performs Transformer distillation at both the pretraining and task-specific learning stages ensures that TinyBERT can capture the general-domain as well as the task-specific knowledge in BERT. [1]

2 Related Work

So far, three main studies have defined the paradigm of model compression via knowledge transfer. The first, by Hinton et al. [2], introduced the seminal concept of Knowledge Distillation (KD), focusing on transferring the so-called dark knowledge from a large Teacher to a compact Student. Their approach minimizes the distance between the output probability distributions of the two networks, softened by a temperature parameter T . The second and more specialized study, by Jiao et al. [1], proposes a variant named TinyBERT, specifically designed for Transformer-based architectures. While Hinton’s method operates on the final output layer, Jiao et al. argue that for complex language models, this is insufficient. Their method leverages a layer-to-layer distillation strategy, forcing the student to mimic not only the teacher’s prediction but also its intermediate representations, such as attention matrices and hidden states. The third, by Ji & Zhu [3], attempts to provide a mathematical justification for why knowledge distillation works, demonstrating that the teacher’s early stopping and the use of hard labels (the truth) are necessary during distillation to counteract the so-called ”imperfect teacher” (i.e., a teacher who makes incorrect predictions). This way, the student can understand how much he can trust the teacher’s predictions.

2.1 Aim of the work

This work has the aim to replicate the distillation strategy proposed by Jiao et al. [1], which consists in two main phases:

1. Phase 1: General Knowledge Distillation. In this phase, the general knowledge learned by BERT (not fine-tuned) during pre-training is transferred, using a large-scale text corpus as data. The student imitates the teacher based on its intermediate layers (Transformer and Embedding), but not on the final prediction it provides.
2. Phase 1: Task-Specific Distillation. The goal of my work is first to develop a teacher specialized in spoiler detection, and then to apply KD to a smaller student model. However, this work also aims to investigate whether using a pre-trained learner on

a generic corpus can avoid phase 1, which can be very costly. Furthermore, using a large dataset can also avoid the data augmentation phase.

In this phase, task-specific knowledge is transferred. BERT is fine-tuned for the task and the student is initialized with the weights obtained in phase 1. Data augmentation is important here: the dataset is augmented because it is often insufficient to achieve good generalization and avoid overfitting. This augmented dataset is then used to have the teacher generate predictions, and the student imitates based on its intermediate layers and logits.

2.2 Data

The dataset used was the IMDB Spoiler Dataset, available on Kaggle. This dataset (in JSON format) contains approximately 570,000 movie reviews from IMDb, each with its own text and a True/False label indicating whether the review contains spoilers or not.

2.2.1 Preprocessing

Given the size of the dataset, I chose to extract a sample of 120,000 total reviews, then divided them into a training set (80%), validation set, and test set (10% each). Since the dataset contains an imbalance in labels (73% False and 26% True),

To overcome the 512 maximum token limit with BERT, a solution based on the head+tail of the review text was used: the text is first tokenized using the BERT vocabulary, also adding the special tokens [CLS] and [SEP] at the end. If the resulting token list is longer than 512, only the first 128 tokens after [CLS] and the last 382 before [SEP] are retained.

Each row, saved in a parquet file for efficiency, contains the list of 512 tokens, an attention_mask of length 512 to highlight to the model that all tokens are important, and the label 0/1.

2.3 Implementation

2.3.1 Structure of the models

2.3.2 Training phase

For the experiments, a desktop PC equipped with a Nvidia RTX 3070 GPU and 32GB of RAM has been used.

2.3.3 Distillation Objectives

3 Results

4 Conclusion

References

- [1] Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., Li, L., Wang, F., Liu, Q.:

- TinyBERT: Distilling BERT for Natural Language Understanding (2020). <https://arxiv.org/abs/1909.10351>
- [2] Hinton, G., Vinyals, O., Dean, J.: Distilling the Knowledge in a Neural Network (2015). <https://arxiv.org/abs/1503.02531>
- [3] Ji, G., Zhu, Z.: Knowledge Distillation in Wide Neural Networks: Risk Bound, Data Efficiency and Imperfect Teacher (2020). <https://arxiv.org/abs/2010.10090>